

Dichotomizing Temporarily Bound Objects Using Capture States

Shuhan Zheng and M. Reza Emami

University of Toronto Institute for Aerospace Studies

ABSTRACT

Temporarily Bound Objects (TBOs) can be classified as Temporarily Captured Orbiters (TCOs) or Flybys (TCFs) depending on whether or not they complete a full revolution around Earth. Given that a TBO's state at the instant of capture contains information of its subsequent class, classification at the early stage of a TBO's capture can reduce the need for prolonged numerical propagation. With a dataset of over 37,000 synthetic TBOs, six classifiers, Naive Bayes Classifier, Probabilistic Neural Network, Logistic Regression, Random Forest, Histogram Gradient Boost, and Multilayer Perceptron are implemented and compared. The strongest classifier, Multilayer Perceptron, reaches 79.50% accuracy on a 3,000-object test set containing equal numbers of TCOs and TCFs. These results demonstrate that the orbital elements at the instant of capture can be useful for potential dichotomization of TBOs.

1. INTRODUCTION

Temporarily Bound Objects (TBOs) are a class of Near-Earth Objects (NEOs) that are temporarily captured by the Earth-Moon System (EMS), thus a negative Keplerian energy with respect to the geocentric inertial frame. In addition, they must reach within 3 Earth Hill radii (EH), with a closest approach to Earth of 1 EH or less. Depending on whether or not a TBO completes the equivalent of a full orbit around Earth in a Sun-Earth geocentric synodic frame during its capture, it can be further differentiated as either a Temporarily Captured Orbiter (TCO) or a Temporarily Captured Flyby (TCF), respectively [1]. Following these definitions, there have been four TBOs of natural origin discovered to date, namely, 2006 RH₁₂₀ [2], 2020 CD₃ [3], 2022 NX₁ [4], and 2024 PT₅ [5]. Among these objects, 2006 RH₁₂₀ and 2020 CD₃ meet the conditions of TCOs [6], while 2022 NX₁ and 2024 PT₅ are TCFs [7, 8].

The astrodynamical characteristics of TCOs place them on the lower end of the geocentric energy spectrum of all the visitors to the EMS, thus requiring minimal energy to redirect them into a stable geocentric orbit [9]. However, a look at recent events involving objects interloping with the EMS shows that the current detection capabilities, albeit rapid advancements in the past two decades, still somewhat fall short of detecting and identifying TBOs sufficiently early to justify a rendezvous mission. For example, 2024 PT₅ was only first discovered by the Asteroid Terrestrial-impact Last Alert System (ATLAS) survey [5] on August 7th, 2024, about a day before it reached its perigee at 1.475 Lunar Distances (LD), and 2020 CD₃ was discovered on February 15th, 2020, after reaching its perigee at 0.122 LD two days earlier [10]. A fundamental conundrum for providing effective early warning for TBOs is in their relatively small sizes (with all four known TBOs being no larger than approximately 15 meters [11, 3, 7, 5]), making them detectable, with sufficient confidence, only by a few advanced telescopes. These telescopes are typically limited in their ability to cover a large enough area of the sky, not to mention their busy schedule for other tasks.

With observation windows for a potential TBO already being short and sparse, establishing a mapping between their arrival conditions and their taxa would reduce the need for telescope time and extensive numerical simulations, enabling rapid scanning and detection for a potentially mission-viable target. Some existing literature, e.g., [12] and [13], establish a population size and orbit distribution of asteroids temporarily captured by the EMS. The work in [13] also expands the NEO population, and modifies the definition of "full orbit" to account for TBOs that may change their orbit rotation. The relevant literature on TBOs generally accepts the synthetic population provided in [13] as representative.

While there exist several investigations about the TBOs origin, size-frequency distribution, and capture dynamics, studies on the distinction between TCOs and TCFs remain limited. This paper explores such distinction from the geocentric inertial osculating orbital elements of TCOs and TCFs at the point of capture, using statistical observations and Machine Learning (ML) techniques, and offers a Multilayer Perceptron (MLP) to assist early dichotomization of an incoming TBO.

2. DATABASE PREPARATION

In [13], a total of 12.5 million initial conditions are sampled from the NEO population distribution outlined in [12], then propagated using a Bulirsch-Stoer n -body integrator with a fixed-step output of three hours for a duration of at least two years, while accounting for gravitational forces from all major bodies, Pluto, and the Moon. (Other perturbations such as solar radiation pressure (SRP) are not accounted for.) Amongst these objects, 174,605 have experienced both negative geocentric Keplerian energy and a minimum geocentric distance of at most 3 EH simultaneously. In the present study, the ephemerides of these objects are re-integrated using the same integrator but with one-hour, fixed-step outputs, and the integration duration is extended one year ahead and after the original ephemerides. Then, by filtering for objects with minimal geocentric distances of less than 1 EH while having a negative Keplerian energy at any point of their trajectory, the ephemeris data of 37,082 objects have been isolated, among which 13,384 are TCOs due to having at least one full orbit around Earth, and the remaining 23,698 are TCFs.

Figure 1 shows histograms of the osculating geocentric inertial orbital elements at the instant of capture, normalized by dividing the number of objects in each bin by the total number of TCOs/TCFs. A bin number of 100 is selected to balance between the histogram’s visual smoothness and visibility of statistical features. Since the probability density of TCOs and TCFs diminishes with decreasing eccentricity, cutoffs are placed at $e = 0.999955$, to exclude numerical regions where the samples are too sparse. The figure also features Probability Density Functions (PDFs) plotted over the histograms. These PDFs are produced using Kernel Density Estimation (KDE) method [14]. Since the definition of a “temporary capture” for a TCO/TCF is based on the geocentric inertial frame, the osculating orbital elements are with respect to this frame, with the exception of Right Ascension of Ascending Node (RAAN) which is defined with respect to the Sun-Earth geocentric synodic frame. This is because “full orbit” around Earth is defined using the cumulative angle measured in the Sun-Earth geocentric synodic frame [13]. Figure 2 shows a comparison between the distributions of Sun-Earth geocentric synodic and inertial RAAN values. It is apparent from the figure that Synodic RAAN, here symbolized as $\Delta\Omega$, is more significant to TCO/TCF delineation.

The KDE method is implemented with `scipy.stats.gaussian_kde`, which is a Python function that represents a kernel-density estimator using Gaussian kernels [15]. The Python function accepts a bandwidth factor via the `bw_method` parameter. The reduced chi-squared metric χ_{red}^2 is calculated between the histogram bin heights and the KDE curve as:

$$\chi_{\text{red}}^2 = \frac{1}{\nu} \sum_{i=1}^{N_{\text{bins}}} \frac{(O_i - E_i)^2}{E_i}; \quad \nu = N_{\text{bins}} - 1 - k \quad (1)$$

where $N_{\text{bins}} = 100$, O_i is the observed histogram counts in the i th histogram bin, E_i is the expected counts given by KDE, and k is the number of fitted parameters, which is 0 in this case. The reduced chi-square metric is widely used to measure the quality of fits, with $\chi_{\text{red}}^2 \approx 1$ indicating an ideal fit, and χ_{red}^2 being greater or less than 1 would indicate under- and over-fit, respectively. The aforementioned bandwidth factor is selected such that $\chi_{\text{red}}^2 \approx 1$.

A test set X_{test} of 3,000 objects (1,500 from TCO and TCF datasets each), has been selected to measure classifier performance. Each classifier’s accuracy is given by the ratio between the number of its correct classifications and the size of the test set.

3. TBO CLASSIFICATION

3.1 Generative (Bayesian-type) Classifiers

With the KDE-generated PDFs that closely represent the population distribution of TCO/TCF osculating orbital elements at capture, the subsequent step is to isolate a set of test objects, re-generate the PDFs with the remaining data, then for each test object compute the following:

$$p(x | y) = \prod_{j=1}^d p_j(x_j | y), \quad d = 6 \quad (2)$$

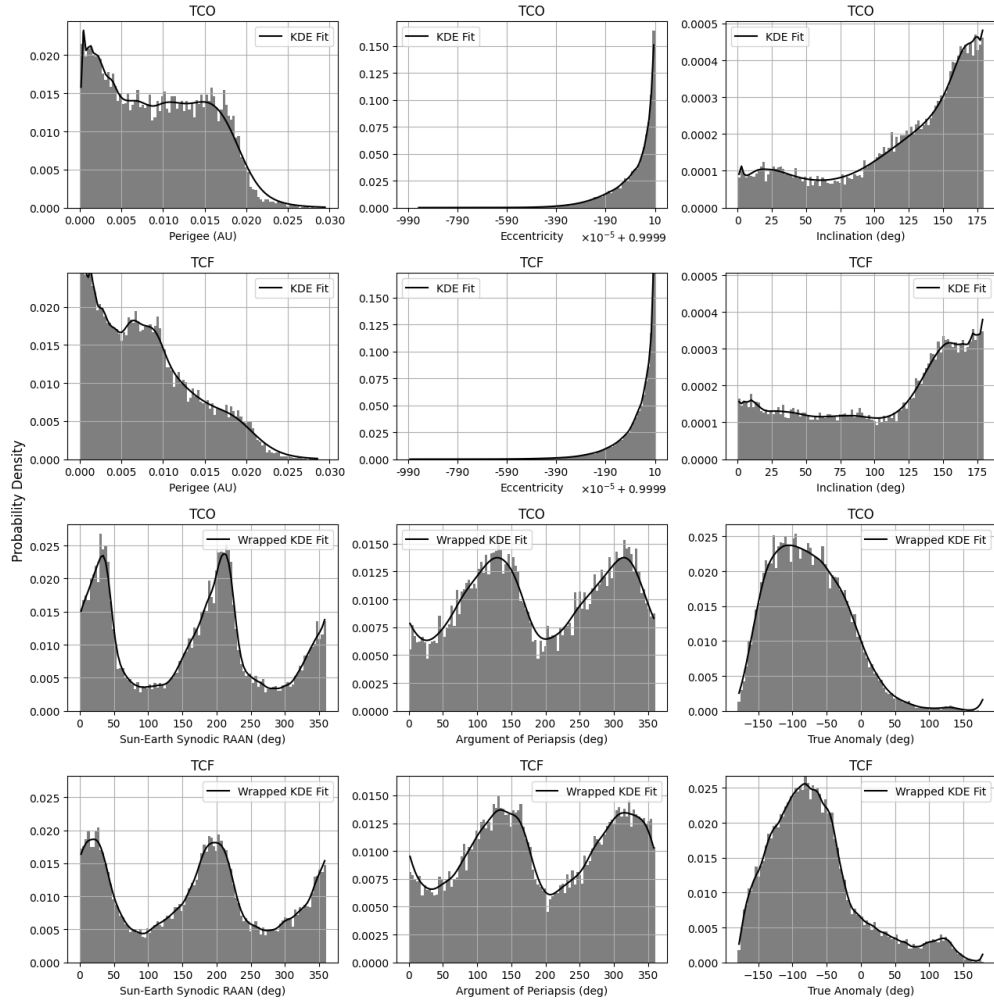


Fig. 1: Probabilistic density of the osculating geocentric inertial orbital elements of TCOs and TCFs at the moment of capture. (RAAN is defined in Sun-Earth geocentric synodic frame.) The black line is a curve-fit over the normalized histograms using the KDE method.

where $x = (x_1, \dots, x_d)$ is the feature vector of the test object, $y \in \{\text{TCO}, \text{TCF}\}$ denotes the class being evaluated, and $p_j(x_j | y)$ is the class-conditional PDF of the j th feature. In the context of TCO/TCF, a feature is an orbital element. Equation (2) is evaluated separately to obtain the probabilities of a test object being a TCO or not, and similarly for TCF. The test object is classified into whichever class with higher probability. The actual implementation is operated in logarithmic space to avoid the numerical instabilities introduced by small $p_j(x_j | y)$ values, thus converting the multiplication in Eq. (2) into addition. This technique is formally recognized as the Naive Bayes Classifier (NBC), which uses Bayes' Theorem to calculate the conditional probability [16]. For the test set X_{test} , an NBC using the remainder of the main dataset as priors yields only 61.70% accuracy.

A core assumption of NBC is that each parameter is statistically independent. The negative geocentric Keplerian energy and 3 EH restrictions place a limit on semi-major axis, which in turn couples perigee radius with eccentricity, through the relationship $q = a(1 - e)$, with q being the perigee radius, a being the semi-major axis, and e being the eccentricity. The resulting poor accuracy may also indicate that the TCO/TCF decision boundary is not adequately captured by the marginal distribution of single orbital elements. The definitions of TCOs and TCFs are inherently multivariate. Cumulative rotation is a fundamental condition that delineates TCOs from TCFs, and for an object with states precisely measured at the instant of capture, its cumulative motion in the subsequent trajectory is pre-determined at that instant by factors including angular momentum and orbital geometry. Therefore, the decision boundary between

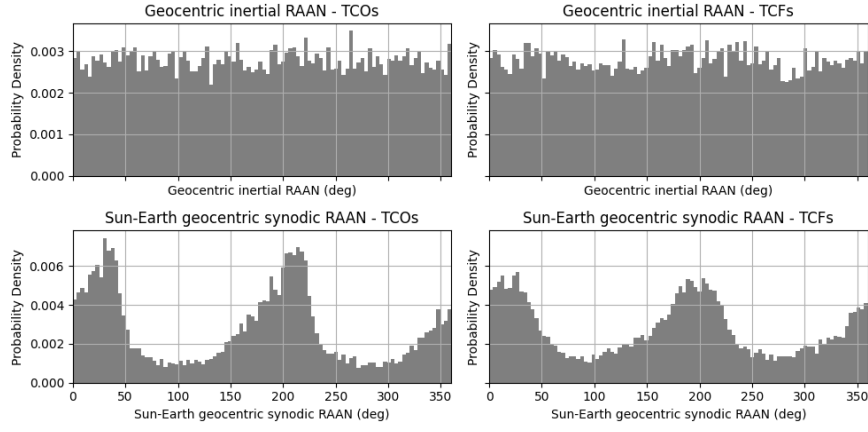


Fig. 2: Comparison between the normalized histograms of geocentric inertial and synodic RAANs.

TCO and TCF, at least in terms of capture states, is necessarily multivariate.

A Probabilistic Neural Network (PNN) relaxes the independence assumption by utilizing multivariate PDFs. PNN is described in [17] as a feed-forward neural network, which is strikingly similar to nonparametric PDF estimators such as KDE. For dichotomizing TCOs and TCFs, a six-feature two-class PNN is implemented in this work. Since all six features have widely differing numerical scales, the input data are rescaled to prevent one feature from unnecessarily biasing the classifier output. The non-angular variables, q , e and i , are rescaled based on their normalized median absolute deviation (MAD), relative to their medians. The differences between the angular variables are evaluated using shortest angular separation. Let $s_n = (s_{n,1}, \dots, s_{n,d})$ denote the feature vector of the n th training object ($n = 1, 2, \dots, N$). Let $\tilde{s}_n$ denote the engineered representation of s_n . For PNN, each feature j is centered by its median m_j and median absolute deviation σ_j^{MAD} across the entire training set, and:

$$\tilde{s}_{n,j} = \frac{s_{n,j} - m_j}{\sigma_j^{\text{MAD}}} \quad (3)$$

where $\sigma_j^{\text{MAD}} = 1.4826 \text{ median}|s_{n,j} - m_j|$. Assign a Gaussian kernel K centered at each $\tilde{s}_n$. Let N_y be the total number of objects in class y , and let S_y be the set of feature vectors of those objects that belong to class y . The class-conditional probability of a test object being in class y is then given by:

$$p(x | y) = \frac{1}{N_y} \sum_{n: \tilde{s}_n \in S_y} K_{\tilde{s}_n, y}(x) \quad (4)$$

The underlying PDF estimator used in PNN is originally proposed in [14], which gives isotropic multivariate Gaussian kernel [17]:

$$K_{\tilde{s}_n, y}(x) = \frac{1}{(2\pi)^{d/2} \sigma_y^d} \exp \left(-\frac{(x - \tilde{s}_n)^2}{2\sigma_y^2} \right) \quad (5)$$

The isotropic assumption holds only when each parameter offers equal contribution to the classifier outputs, but that is not the case for TCOs and TCFs. In [18], it is established that i) Jovian irregular satellites originate from captured satellites; and ii) such irregular satellites tend to occupy highly inclined, retrograde orbits. This tendency is also observed among TCOs and TCFs, as shown in Figure 1. Since TCOs have a potential to become a permanent capture, the synthetic inclination distributions in the present study, together with the surveys of Jovian irregular satellite orbits, suggest that the geocentric inclination may have a stronger influence on the number of revolutions completed during a temporary capture than the other orbital elements considered here. Figure 1 also shows that different orbital elements can offer varying degrees of differentiation between TCOs and TCFs. Therefore, the PNN-based classifier must utilize anisotropic Gaussian kernels, as:

$$K_{\tilde{s}_n, y}(x) = \frac{1}{(2\pi)^{d/2} \prod_{j=1}^d \sigma_{y,j}} \exp \left[-\frac{1}{2} \sum_{j=1}^d \left(\frac{x_j - \tilde{s}_{n,j}}{\sigma_{y,j}} \right)^2 \right] \quad (6)$$

where $\sigma_{y,j}$ is the width factor for the j th orbital element and class y . Also, since the class-conditional density scores of TCOs and TCFs, p_{TCO} and p_{TCF} , are evaluated individually, the TCO/TCF decision boundary can be biased by setting a threshold value D , such that a test object is classified as a TCO if $p_{\text{TCO}} - p_{\text{TCF}} \geq D$; otherwise, it is a TCF. Two separate validation sets of 500 objects are selected from the remainder of TCO and TCF datasets, in order to find values of σ and D to achieve the model's near-maximum accuracy. With $D = 0.11$ and width factors σ listed in Table 1, an accuracy of 68.93% is obtained on the test set. The increased accuracy, compared to that of NBC (61.70%), validates the multivariate feature regarding the TCO/TCF decision boundary.

Table 1: Width factors σ for each orbital element of TCO and TCF

Class	Perigee (q)	Eccentricity (e)	Inclination (i)	Sun-Earth geocentric synodic RAAN ($\Delta\Omega$)	Argument of Perigee (ω)	True Anomaly (θ)
TCO	0.2205	0.1995	0.2100	0.2153	0.2100	0.2100
TCF	0.1805	0.1853	0.1853	0.1853	0.1948	0.1948

While PNN is robust as a nonparametric classifier, it compares each test object against the entire stored training set through exhaustive kernel comparison, until a decision boundary spontaneously emerges, which is then refined through tuning σ . The model's size grows proportionally with the size of the training set, with computation time growing along with them, imposing a linearly increasing cost for accuracy gains. Such exhaustive kernel comparison must be repeated for each test object, without retaining any compact list of parameters, in order to make the decision about a test object.

3.2 Discriminative Classifiers

While generative classifiers attempt to build the joint distribution model $p(x, y)$, an alternative approach known as discriminative classifiers, aims to learn $p(x|y)$ directly, or in some cases a decision boundary [19].

Note that among the orbital elements outlined in Table 1, $\Delta\Omega$, ω , and θ are periodic variables, thus their scalar numerical representations will have a discontinuity at 0° and 360° . The implemented generative models handle this discontinuity by computing circular distances and using wrapped Gaussian kernels. However, since the implemented discriminative models would otherwise consider the angles as scalar numerical inputs, the periodic variables are decomposed into their sine and cosine components, resulting in a total of nine features. In the case of discriminative classifiers, $\tilde{s}_n$ denotes the set of nine engineered features.

Four discriminative classifiers, namely, Logistic Regression (LR), Random Forest (RF), Histogram Gradient Boosting (HGB), and Multilayer Perceptron (MLP), are implemented in Python and examined below. Among these models, LR provides a linear baseline for comparison, RF and HGB offer a comparison between two different nonlinear decision tree-based models, as well as a reevaluation of the results in [1] with an expanded dataset, and MLP provides an alternative architecture as a nonlinear neural-network classifier. Each of these classifiers is subjected to a five-fold cross-validation step, in which the training data is split into five roughly equal-sized partitions. Each partition serves as validation set while the union of the other four remain a training set, thus forming five individually trained models. The outputs of these models are then aggregated through either a majority vote or by the average of predicted probabilities from all five.

3.2.1 Logistic Regression

In LR, the decision boundary is defined by a linear hyperplane, which is the zero level set of the predictor. This hyperplane is formed by finding a linear combination of all nine features. Let the coefficients of the linear combination be elements of $\beta = [\beta_1, \dots, \beta_d]^T$ ($d = 9$), and the level set $z = 0$ be the decision hyperplane. Then, for the n th object in the training set with feature vector $\tilde{s}_n$, the linear predictor can be written as:

$$z_n = \beta_0 + \sum_{j=1}^d \beta_j \tilde{s}_{n,j} \quad (7)$$

where β_0 indicates the predictor's bias. The absolute value of z_n correlates to the object's degree of separation from the decision boundary, and the sign of z_n indicates which side of the decision boundary the object occupies. Since z_n is an unbounded value, a sigmoid function is used to bound it between 0 and 1. This sigmoid function maps the linear predictor to the modeled probability of the positive TCF class for the n th training object, and it is written as:

$$p_{\text{TCF},n} = \frac{1}{1 + \exp(-z_n)} \quad (8)$$

Rearranging Eq. (8) and taking the natural logarithm gives the classifier's log-odds function:

$$\ln \left(\frac{p_{\text{TCF},n}}{1 - p_{\text{TCF},n}} \right) = \beta_0 + \sum_{j=1}^d \beta_j \tilde{s}_{n,j} \quad (9)$$

Equation (9) shows that LR models the log-odds of the positive-class probability as a linear function of $\tilde{s}_n$. Define a binary target value t_n , such that $t_n = 0$ indicates that the n th training object is a TCO, and 1 for TCF:

$$\begin{aligned} p_{\text{TCF},n} &= P(t_n = 1 \mid \tilde{s}_n, \beta, \beta_0), \\ 1 - p_{\text{TCF},n} &= P(t_n = 0 \mid \tilde{s}_n, \beta, \beta_0). \end{aligned} \quad (10)$$

The Bernoulli distribution of the n th training object's contribution can, therefore, be written as:

$$p_{\text{TCF},n}^{t_n} (1 - p_{\text{TCF},n})^{1-t_n} = \begin{cases} p_{\text{TCF},n}, & t_n = 1, \\ 1 - p_{\text{TCF},n}, & t_n = 0. \end{cases} \quad (11)$$

Since each training object is independently propagated from a distribution, the joint likelihood of N objects is:

$$L(\beta_0, \beta) = \prod_{n=1}^N p_{\text{TCF},n}^{t_n} (1 - p_{\text{TCF},n})^{1-t_n} \quad (12)$$

Taking the negative logarithm of Eq. (12) gives the binary cross-entropy objective $E(\beta_0, \beta)$ [20]:

$$E(\beta_0, \beta) = -\ln L = -\sum_{n=1}^N [t_n \ln p_{\text{TCF},n} + (1 - t_n) \ln(1 - p_{\text{TCF},n})] \quad (13)$$

The LR method then seeks to minimize Eq. (13) by perturbing each element of β . The implementation uses the default Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) solver offered by the Python function `Logistic Regression` from the `sklearn.linear_model` package. The implemented model additionally applies the default L_2 regularization [21].

For both aggregation schemes, LR results in an accuracy of 50% with the test set. The random performance signifies that the model is inadequate for predicting whether a test object is a TCO or TCF. Therefore, it can be concluded that a linear hyperplane, formed by a linear combination of capture states, is inadequate to reliably delineate the TCO and TCF populations. In addition, the failure of LR in TCO/TCF delineation indicates that the decision boundary between TCO and TCF is inherently nonlinear. This is consistent with the physical understanding for the nonlinear dynamics of temporarily bound objects.

3.2.2 Random Forest

The LR method produces a single decision boundary that separates the positive and negative classes, thus lacking the flexibility to represent a nonlinear decision boundary. A common strategy to approach nonlinearity is to approximate a nonlinear problem as a collection of many infinitesimal linear partitions. In the context of TCO/TCF dichotomization, decision tree-based methods, namely RF and HGB, recursively introduce infinitesimal binary partitions within the state

space, before a local linear decision boundary is established in each partition [22]. With a sufficiently small partition, a local decision boundary can be established by simply applying thresholds in multiple state space dimensions, and a global nonlinear decision boundary emerges from many of these local linear decision boundaries.

Let $R \subseteq \{1, \dots, N\}$ be a set of indices of training objects. For class $y \in \{\text{TCO}, \text{TCF}\}$, let $N_{R,y}$ denote the number of objects in R belonging to class y . The probability of randomly selecting an object from R with class y is:

$$p_y(R) = \frac{N_{R,y}}{N_R} \quad (14)$$

where N_R is the number of objects in R . Then, the Gini impurity is defined as [22]:

$$G(R) = 1 - \sum_{y \in \{\text{TCO}, \text{TCF}\}} p_y(R)^2 \quad (15)$$

The RF method splits R into daughter partitions R_0 and R_1 by applying a threshold $v \in V_j(R)$ to a feature with index $j \in J_R$, where V_j is a set of candidate thresholds, and $J_R \subseteq J$ is the set of features in $J = \{1, \dots, d\}$ ($d = 9$). Taking a weighted average of $G(R_0)$ and $G(R_1)$ gives the expected impurity after the split, which is denoted as G_{split} :

$$G_{\text{split}}(R, j, v) = \frac{N_{R_0}}{N_R} G(R_0) + \frac{N_{R_1}}{N_R} G(R_1) \quad (16)$$

where N_{R_0} and N_{R_1} are the numbers of objects in R_0 and R_1 , respectively. The difference between $G_{\text{split}}(R, j, v)$ and $G(R)$ defines the impurity reduction $\Delta G(R, j, v)$

$$\Delta G(R, j, v) = G(R) - G_{\text{split}}(R, j, v) \quad (17)$$

Each partition R is divided along a hyperplane defined by $\tilde{s}_{n,j^*} = v^*$. The pair (j^*, v^*) is a unique pair of a feature and its threshold value that maximizes the Gini impurity reduction within the partition R , and this pair is formally written as:

$$(j^*, v^*) = \arg \max_{\substack{j \in J_R \\ v \in V_j(R)}} \Delta G(R, j, v) \quad (18)$$

and the resulting daughter partitions of R are given by:

$$\begin{aligned} R_0 &= \{\forall n \in R : \tilde{s}_{n,j^*} \leq v^*\}, \\ R_1 &= \{\forall n \in R : \tilde{s}_{n,j^*} > v^*\}. \end{aligned} \quad (19)$$

The same process is then repeated for all subsequent daughter partitions, until $G(R) = 0$ or when the number of objects N_R in R is below a pre-defined minimal value $N_{R,\min}$. The default value of $N_{R,\min} = 2$ is used. This method of selecting features and thresholds to maximally reduce impurity through recursive partitioning of state space follows the procedure outlined in [22]. Given that a decision tree represents only one specific sequence of threshold choices made using the training objects, its outcome strongly depends on that sequence.

The RF method stabilizes the output by reducing the correlation between decision trees, through a process known as “bootstrapping” [23]. Bootstrapping randomly selects objects from the training set, such that some objects may be selected more than once, and some not at all, thus preserving the overall sample count. The subset J_R is also randomized to further reduce the inter-tree correlation.

Let $R_b(x)$ be the terminal partition reached by a test object x on decision tree b . The probability of TCFs in $R_b(x)$, namely, $p_b(x)$, can therefore be written as:

$$p_{\text{TCF},b}(x) = \frac{1}{N_{R_b}} \sum_{n \in R_b(x)} t_n \quad (20)$$

where N_{R_b} is the total number of objects in R_b , and the binary target value t_n is defined by Eq. (10). Conversely, the probability of TCOs in $R_b(x)$ is $1 - p_b(x)$. This produces a series of TCO and TCF probability for each decision tree, which is then aggregated by `RandomForestClassifier`, a Python function in the `sklearn.ensemble` library [24], through averaging all TCO and TCF probability across all decision trees. For a total of B decision trees, the resulting TCF probability is given by:

$$p_{\text{TCF}}(x) = \frac{1}{B} \sum_{b=1}^B p_{\text{TCF},b}(x) \quad (21)$$

and since TCO is the complement of TCF, the following expression can be established for RF's final prediction $\hat{t}(x)$ of a test object x :

$$\hat{t}(x) = \begin{cases} 1, & p_{\text{TCF}}(x) > D, \\ 0, & p_{\text{TCF}}(x) \leq D. \end{cases} \quad (22)$$

where D is a decision threshold that can be adjusted to maximize accuracy.

3.2.3 Histogram Gradient Boosting

In the RF method, each decision tree is trained independently from others, and their collective variance is reduced through averaging their outcomes. Therefore, there is no mechanism through which the individual decision trees can inform each other's construction. The HGB method, on the other hand, constructs the decision trees sequentially, such that the decision trees constructed later progressively refine deficiencies left by the previous trees.

Let $m = 0, 1, \dots, M$ denote the index of the HGB iteration, and define $F_m(\tilde{s}_n)$ as the classifier's raw output for the n th training object's feature $\tilde{s}_n$ after m iterations. Each iteration in the recursion can be abstractly expressed as:

$$F_m(\tilde{s}_n) = F_{m-1}(\tilde{s}_n) + \eta H_m(\tilde{s}_n) \quad (23)$$

where $H_m(\tilde{s}_n)$ is the contribution of the current tree, in the form of numerical updates to the current prediction. $H_m(\tilde{s}_n)$ is multiplied by a learning rate η , which moderates the contribution of each tree [25]. After M iterations, Eq. (23) becomes

$$F_M(\tilde{s}_n) = F_0 + \eta \sum_{m=1}^M H_m(\tilde{s}_n) \quad (24)$$

with F_0 being the initial baseline prediction, and F_M being the raw output of the classifier constructed at the M th and final HGB iteration. Since $F_M(\tilde{s}_n) \in \mathbb{R}$ is unbounded, it must be mapped to $p_{\text{HGB}} \in [0, 1]$ for binary classification. The implementation uses the default settings of the Python function `HistGradientBoostingClassifier` from the `sklearn.ensemble` package, which utilizes the logistic sigmoid to obtain the positive-class probability, as shown below [26]:

$$p_{\text{TCF}}(\tilde{s}_n) = \frac{1}{1 + \exp[-F_M(\tilde{s}_n)]} \quad (25)$$

The complementary TCO probability is then

$$p_{\text{TCO}}(\tilde{s}_n) = 1 - \frac{1}{1 + \exp[-F_M(\tilde{s}_n)]} \quad (26)$$

The form of Eq. (25) is reminiscent of Eq. (8). Therefore, the binary cross-entropy objective for HGB can be obtained through the same steps as those for LR, except that HGB seeks the $H_m(\tilde{s}_n)$ such that the current objective is reduced.

For N training objects, the binary cross-entropy objective, which is the total training loss produced by the $(m - 1)$ trees ensemble, can be expressed as

$$E_{m-1} = - \sum_{n=1}^N \left[t_n \ln p_{\text{TCF}}^{(m-1)} + (1 - t_n) \ln (1 - p_{\text{TCF}}^{(m-1)}) \right] \quad (27)$$

where $p_{\text{TCF}}^{(m-1)}$ is the positive probability before constructing the m th decision tree, and the binary target value t_n is defined by Eq. (10). Computing the derivative $\frac{\partial E_{m-1}}{\partial F}$ gives the gradient, negative of which identifies the direction from F_{m-1} that reduces the loss [25]. In the implementation, Newton boosting is used by `sklearn` to solve for $H_m(\tilde{s}_n)$ using the first and second derivatives [27].

Similarly to LR, the implementations of RF and HGB utilize the five-fold cross-validation procedure, and aggregate the respective predictions through a voting and an averaging scheme. This yields the best accuracies of 75.30% for RF with majority voting and 72.10% for HGB with averaging.

3.2.4 Multilayer Perceptron

Although RF and HGB both produce decision boundaries that appear globally nonlinear and multivariate, these decision boundaries are ultimately composed of the axis-aligned faces of many smaller hyper-rectangles. Significant improvements in model accuracy may require abandoning this hyper-rectangular approximation for greater representational flexibility. For example, the aforementioned coupling between q and e becomes a linear hyperplane through the origin for fixed value a , but will manifest as a corrugated surface when represented by hyper-rectangles instead.

The MLP method relaxes the hyper-rectangular assumption by constructing its decision boundary through simultaneous considerations of multiple features. The implementation contains three hidden layers between the nine-feature input layer and the two-class output layer, with the output layer represented by a single neuron, which returns a scalar output. From input to output, the first, second and third hidden layer contains 128, 64, and 32 neurons, respectively. Each neuron computes an affine combination of the outputs from all neurons in the preceding layer, with learned weights and a bias term. For the hidden layers, the outcome of this affine combination is then applied with an activation function [20]. In the implementation with `Pytorch` [28], the activation function of each hidden layer's neurons is the Rectified Linear Unit (ReLU) function [29]:

$$\phi(u) = \max(0, u) \quad (28)$$

where u is the affine pre-activation, and ϕ is the activation function. The ReLU function preserves u unless it is negative, in which case it is set to 0. Since the composition of affine combinations is also an affine combination, ReLU activation functions prevent the layers from collapsing algebraically, preserving the network's representational flexibility.

The five-fold cross-validation method is also utilized in the MLP implementation. Since MLP is trained through gradient-based optimization, it is sensitive to adverse effects of input features having widely different numerical scales. Therefore, its nine input features are standardized using their mean and standard deviation. For each of the five cross-validation segments, these values are computed from the training subset, before applying to the corresponding training, validation, and test sets. Following the same derivation outlined in Eqs. (9) through (13), the binary cross-entropy of MLP can be derived from the sigmoid function computed from the output the final neuron z_n .

The MLP's loss is minimized by updating the network parameters using the Adam algorithm [30] with a configured learning rate of 10^{-4} and batch size of 128. During training, the implementation monitors the model's validation accuracy at each training epoch. After a 20-epoch grace period, training is stopped if the validation accuracy has failed to improve for 60 consecutive epochs, and the network parameters corresponding to the best validation accuracy are retained. With the 3,000-object test set, MLP yields an accuracy of 79.27% with majority voting, and 79.50% with averaging.

3.3 Classifier Evaluation and Decision Threshold Selection

A confusion matrix is constructed to summarize the predictions versus the actual labels provided by the five-model ensemble, presented in (Table 2). The matrix is constructed on the basis that the positive class represents the majority in the training set, and the negative class represents the minority. The positive/negative definition used here is opposite from the definitions in [1], since this study has introduced a larger set of data points, in which TCF becomes the majority instead.

Table 2: Confusion matrix		
	Actual TCF	Actual TCO
Predicted TCF	True Positive (TP)	False Positive (FP)
Predicted TCO	False Negative (FN)	True Negative (TN)

The confusion matrix facilitates the calculation of the following evaluation metrics, which are adopted from [1]:

- **Accuracy:** the proportion of correct TCO and TCF predictions within the overall number of predictions, evaluates the model's overall performance.

$$\text{Accuracy} = \frac{TP + TN}{TN + FP + TP + FN} \quad (29)$$

- **Precision:** the proportion of correctly predicted positive within the total number of positive predictions, evaluates the model's confidence in predicting the positive class. In the TCO/TCF context, it measures the ratio of TCF predictions that are indeed TCFs.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (30)$$

- **Recall:** the proportion of correctly predicted positives within the total number of actual positives, indicating the model's probability of correctly predicting the positive class. In the TCO/TCF context, it measures the ratio of TCFs that are correctly identified as TCFs.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (31)$$

- **Specificity:** the proportion of correctly predicted negatives within the total number of actual negatives, indicating the model's probability of correctly predicting the negative class. In the TCO/TCF context, it measures the ratio of TCOs being correctly identified as TCOs.

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (32)$$

- **Negative Predicted Value (NPV):** the ratio of accurately predicted negatives to the overall number of negative predictions, indicating the model's confidence in predicting the negative class. In the TCO/TCF context, it measures the ratio of TCO predictions that are indeed TCOs.

$$\text{NPV} = \frac{TN}{TN + FN} \quad (33)$$

- **f1-score:** a value between 0 and 1 measuring the balance between precision and recall.

$$\text{f1-score} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (34)$$

Since each discriminative classifier produces a continuous class-probability score, a decision threshold D is applied such that a test object is classified as a TCF if $p_{\text{TCF}} \geq D$, and a TCO if otherwise. Previous works, such as [1], have shown that models can have substantially differing degrees of performance for each class. Therefore, instead of setting $D = 0.5$, the implementation iterates through a discrete grid of D between 0 and 1. For each candidate D , the accuracy of each segment is measured with its corresponding validation subset, then these accuracies are aggregated through weighted averaging, with the weights being the size of each validation subset. The determined values of the decision threshold for discriminative classifiers are listed in Table 3. It is worth mentioning that the values in the table are selected to maximize aggregate validation accuracy, which implicitly assumes that the two TCO and TCF detection errors are equally costly. In practice this may not be true. A false TCF, that is, a genuine TCO discarded, forfeits a rendezvous opportunity that may not recur for years, whereas a false TCO costs a few hours of telescope time and a short arc of astrometry. Raising D above the accuracy-optimal 0.495 shifts marginal objects into the TCO class, trading precision for recall on the class that matters operationally. Because the classifier emits a continuous score, this trade can be made per campaign rather than once and for all, and the appropriate operating point should be selected from the cost ratio of the specific follow-up mission.

4. RESULTS AND DISCUSSION

Table 3 summarizes the performance of the classifiers presented in this paper in identifying a synthetic database of 37,082 TBOs as either a TCO or TCF, using merely their state vector at the capture point. Note that the aggregation methods and decision threshold are not applicable to generative classifiers, because these classifiers are implemented without cross-validation, and the decision threshold for PNN controls the difference between p_{TCO} and p_{TCF} , rather than the $D \in [0, 1]$ threshold implemented for the discriminative classifiers.

The MLP method emerges as the strongest classifier by every reported metric, reaching 79.50% accuracy on the 3,000-object balanced test set. Aside from LR, which yields no better than chance and is retained only as a linear baseline, averaging the class probabilities of the five cross-validation segments seems to be a better aggregation scheme for discriminative classifiers overall, RF being the sole exception. For both HGB and MLP, averaging outperforms majority voting, which is expected: majority voting discards the magnitude of each segment’s confidence, and retains only the sign of $(p_{\text{TCF}} - D)$, whereas averaging preserves the ensemble’s graded assessment of objects that lie close to the decision boundary. Since a substantial fraction of the population does lie close to that boundary, the information discarded by voting is precisely the information that matters most.

The ordering of the accuracy of four discriminative models, $\text{LR} \ll \text{HGB} < \text{RF} < \text{MLP}$, is itself informative and can be interpreted dynamically. Indeed, the apparent failure of LR is more instructive than its accuracy value suggests. As pointed out in Section 3.1, the relation between the feature pair (q, e) and semi-major axis a is hyperbolic, not linear. Therefore, in the (q, e) plane the level sets of constant a form a pencil of straight lines through the singular point $(q, e) = (0, 1)$, so no single hyperplane in (q, e) can separate small- a from large- a objects. The LR method is structurally incapable of representing this relationship, and its collapse to the majority-class prediction (recall = 1.0000, specificity = 0.0000) is an expected consequence. The decision-tree ensembles recover the relationship, but only by tiling it. The RF and HGB methods approximate the pencil of constant- a lines with axis-aligned hyper-rectangles, producing the corrugated boundary noted in Section 3.2.4. The cost of such approximation grows steeply as $e \rightarrow 1$, because $\frac{\partial a}{\partial e} = \frac{q}{(1-e)^2}$, which diverges at the parabolic limit. At the eccentricity cutoff adopted in Section 2, $e = 0.999955$, a perigee radius of one lunar distance implies $a \approx 8.5 \times 10^9$ km, some four orders of magnitude beyond the Hill radius. In this regime the osculating semi-major axis has essentially no dynamical meaning: at $r \simeq 1$ EH the solar tidal acceleration equals the geocentric attraction by construction, the two-body osculating elements vary on the same timescale as the motion itself, and the geocentric orbit is better described as a transit trajectory through the Sun–Earth L_1/L_2 gateway than as a perturbed ellipse [31]. A finite fraction of the training population, therefore, occupies a region of feature space where the selected coordinates are intrinsically ill-conditioned, and no classifier operating on those coordinates can be expected to separate the classes there. The advantage of MLP is that its affine-plus-ReLU compositions can bend a single boundary surface along the $q/(1-e)$ ridge instead of tiling across it, which is consistent with its gain of roughly four percentage points over RF at equal input information.

Every nonlinear model exhibits a recall notably larger than its specificity, by 20 to 27 percentage points. With TCF as the positive class, this means the classifiers identify actual TCFs considerably more reliably than actual TCOs. For the best model (MLP with averaging), the 615 misclassifications of the balanced test set decompose into 462 TCOs

Table 3: Performance comparison of classification models

Model	Aggregation Method	Threshold	Accuracy	Precision	Recall	Specificity	NPV	F1
Naive Bayes Classifier	N/A	N/A	0.6170	0.6139	0.6307	0.6033	0.6203	0.6222
Probabilistic Neural Network	N/A	N/A	0.6894	0.6768	0.7247	0.6540	0.7037	0.6999
Logistic Regression	Majority Vote	0.210	0.5000	0.5000	1.0000	0.0000	0.0000	0.6667
	Averaging		0.5000	0.5000	1.0000	0.0000	0.0000	0.6667
Random Forest	Majority Vote	0.515	0.7530	0.7024	0.8780	0.6280	0.8373	0.7804
	Averaging		0.7464	0.6956	0.8760	0.6167	0.8326	0.7754
Histogram Gradient Boost	Majority Vote	0.535	0.7190	0.6732	0.8513	0.5867	0.7978	0.7518
	Averaging		0.7210	0.6744	0.8547	0.5873	0.8016	0.7539
Multilayer Perceptron	Majority Vote	0.495	0.7927	0.7417	0.8980	0.6873	0.8708	0.8124
	Averaging		0.7950	0.7446	0.8980	0.6920	0.8715	0.8141

labeled as TCFs and only 153 TCFs labeled as TCOs; three quarters of all errors are, therefore, of a single type. This asymmetry is systematic across RF, HGB and MLP, and at the selected decision thresholds it is not an artifact of any one architecture. However, more reliable TCF identification can also be attributed to their larger number in the training data set, which consists of 20,698 TCFs and 10,384 TCOs. The dominance of TCFs in the TBO population, nonetheless, is a natural consequence of n -body orbital dynamics that are implemented for generating the synthetic database. Note that the metrics in Table 3 are computed on a deliberately balanced test set, and therefore describe the classifier. In the synthetic population the class prior is 36.1% TCO and 63.9% TCF. Applying the MLP’s class-conditional rates at that prior gives an expected accuracy of 82.4%, a positive predictive value for TCF of 83.8%, and, most relevantly for mission planning, a negative predictive value for TCO of 79.3%. In other words, an object that the classifier labels a TCO is a genuine TCO about four times in five, against a base rate of roughly one in three: the classifier delivers a 2.2-times enrichment of the TCO fraction in a follow-up queue. Given that a TCO offers a residence time of months rather than days and sits at the lowest end of the geocentric energy spectrum of all visitors to the EMS [9], this enrichment is directly convertible into observing-time savings. Also, TCOs have longer residence times and smaller perigees, so they are more detectable; Hence, discovery may be further biased toward them.

The systematic asymmetry between recall and specificity may be explained through label marginality, as well. Objects whose cumulative angle falls just above or just below 2π are dynamically indistinguishable but are assigned to opposite classes, and it is reported in [1] that TCO/TCF transitions can be induced by numerical error in the revolution count alone. The population near the boundary is therefore split by a threshold that carries no dynamical content. This mechanism also supplies the second explanation for the recall/specificity asymmetry. Because the training set contains nearly twice as many TCFs as TCOs, a marginal object falling in the ambiguous region is more likely to be assigned the majority label, so marginal TCOs are preferentially absorbed into the TCF class. The first explanation, consistent with [13], is that below 4 lunar distances (roughly equivalent to 1 EH) the geocentric speed is more tightly constrained for TCFs than for TCOs; although geocentric speed is not an explicit feature here, it is collectively encoded by q , e and θ , so a more constrained speed distribution translates directly into a more compact and more learnable region of feature space. The two explanations are complementary: TCFs are both intrinsically more concentrated and favored by the prior.

The classification results confirm that the capture state of a temporarily bound object carries substantial predictive information about its subsequent evolution into a TCO or a TCF. Motion through the Earth–Moon system under the gravitational influence of all major bodies, the Moon and Pluto is deterministic, so in principle the entire future trajectory is fixed by the state at capture; but it is also chaotic, and the map from capture state to outcome is correspondingly convoluted. The fact that a machine learning model can recover 79.50% of that map without any explicit representation of the underlying physics is a key result of this paper. Equally important is what the residual 20.5% represents, since a portion of it is not reducible by a better classifier.

Some limitations bound the present results. First, the classifier assumes the state is known exactly at the exact instant of capture, which real observations rarely provide; if a candidate is discovered after the capture epoch, as was the case for both 2020 CD₃ and 2024 PT₅, numerical propagation remains the more convenient route to dichotomization. Secondly, the feature set omits several quantities that can also be dynamically vital for the classification, notably the

osculating semi-major axis itself, the sign of the geocentric angular momentum projection onto the synodic rotation axis, the Sun–Earth Jacobi constant, and the Earth–Moon geometry at capture. Thirdly, the labels themselves are approximate near the one-revolution boundary. Accordingly, the most promising extensions are to propagate realistic orbit-determination uncertainties through the capture state and the capture epoch to test whether the classification remains robust; to supply the classifier with physically motivated derived features, in particular semi-major axis in the synodic frame, and the Jacobi constant, which would relieve the network of having to reconstruct the relation between the perigee radius, eccentricity, and semi-major axis; to re-cast the problem as a regression on cumulative synodic angle, with the TCO/TCF label recovered by thresholding, so that marginal objects contribute their true information content rather than a noisy binary; and to admit repeated-capture objects as a distinct third class.

5. REFERENCES

- [1] Sean Wolfe and M. Reza Emami. Temporary Captures in Earth-Moon System: A Taxonomy Design using Machine Learning. *The Journal of the Astronautical Sciences*, 71(6):52, November 2024. ISSN 2195-0571. doi: 10.1007/s40295-024-00473-4. URL <https://doi.org/10.1007/s40295-024-00473-4>.
- [2] Tomasz Kwiatkowski, Agnieszka Kryszczyńska, Magdalena Polinska, David Buckley, Darragh O'Donoghue, Phil Charles, Lisa Crause, Steven Crawford, Yas Hashimoto, Alexei Kniazev, et al. Photometry of asteroid 2006 rh120 during its short visit to a geocentric orbit. *Asteroids, Comets, Meteors 2008*, 1405:8297, 2008.
- [3] Bryce T. Bolin, Christoffer Fremling, Timothy R. Holt, Matthew J. Hankins, Tomás Ahumada, Shreya Anand, Varun Bhalariao, Kevin B. Burdge, Chris M. Copperwheat, Michael Coughlin, et al. Characterization of temporarily captured minimoon 2020 cd3 by keck time-resolved spectrophotometry. *The Astrophysical Journal Letters*, 900(2):L45, 2020.
- [4] Carlos de la Fuente Marcos and Raúl de la Fuente Marcos. How to become a mini-moon: Some hints from 2022 nx1. *Research Notes of the AAS*, 6(8):160, August 2022. doi: 10.3847/2515-5172/ac8809. URL <https://dx.doi.org/10.3847/2515-5172/ac8809>.
- [5] Carlos de la Fuente Marcos and Raúl de la Fuente Marcos. A two-month mini-moon: 2024 pt5 captured by earth from september to november. *Research Notes of the AAS*, 8(9):224, September 2024. doi: 10.3847/2515-5172/ad781f. URL <https://dx.doi.org/10.3847/2515-5172/ad781f>.
- [6] Grigori Fedorets, Marco Micheli, Robert Jedicke, Shantanu P. Naidu, Davide Farnocchia, Mikael Granvik, Nicholas Moskovitz, Megan E. Schwamb, Robert Weryk, Kacper Wierzchoś, et al. Establishing earth's minimoon population through characterization of asteroid 2020 cd3. *The Astronomical Journal*, 160(6):277, 2020.
- [7] R. de la Fuente Marcos, J. de León, C. de la Fuente Marcos, J. Licandro, M. Serra-Ricart, and A. Cabrera-Lavers. Mini-moons from horseshoes: A physical characterization of 2022 nx1 with osiris at the 10.4 m gran telescopio canarias. *Astronomy and Astrophysics*, 670:L10, February 2023. ISSN 1432-0746. doi: 10.1051/0004-6361/202245514. URL <http://dx.doi.org/10.1051/0004-6361/202245514>.
- [8] R. de la Fuente Marcos, J. de León, M. Serra-Ricart, C. de la Fuente Marcos, M. R. Alarcon, S. Geier, A. Tejero, A. Pérez Romero, J. Licandro, and A. Cabrera-Lavers. Basaltic quasi-mini-moon: Characterizing 2024 pt5 with the 10.4 m gran telescopio canarias and the two-meter twin telescope. *Astronomy and Astrophysics*, 694:L5, 2025. doi: 10.1051/0004-6361/202452713.
- [9] Hodei Urrutxua, D. Scheeres, Claudio Bombardelli, Juan Luis Gonzalo, and Jesus Pelaez. What does it take to capture an asteroid? a case study on capturing asteroid 2006 rh120. *Advances in the Astronautical Sciences*, 152: 1117–1136, January 2014.
- [10] NASA Jet Propulsion Laboratory. Small-body database lookup: 2020 cd3. URL https://ssd.jpl.nasa.gov/tools/sbdb_lookup.html. JPL Solar System Dynamics Small-Body Database.
- [11] Brent W. Barbee and Paul W. Chodas. Near-earth asteroids 2006 rh120 and 2009 bd: Proxies for maximally accessible objects? In *2015 AAS/AIAA Astrodynamics Specialist Conference*, number GSFC-E-DAA-TN25192, 2015.
- [12] Mikael Granvik, Jeremie Vaubaillon, and Robert Jedicke. The population of natural earth satellites. *Icarus*, 218 (1):262–277, 2012. ISSN 0019-1035. doi: 10.1016/j.icarus.2011.12.003. URL <https://www.sciencedirect.com/science/article/pii/S0019103511004684>.
- [13] Grigori Fedorets, Mikael Granvik, and Robert Jedicke. Orbit and size distributions for asteroids temporarily captured by the earth-moon system. *Icarus*, 285:83–94, 2017. ISSN 0019-1035. doi: 10.1016/j.icarus.2016.12.022. URL <https://www.sciencedirect.com/science/article/pii/S0019103516306480>.
- [14] Emanuel Parzen. On estimation of a probability density function and mode. 33(3):1065–1076. doi: 10.1214/aoms/1177704472.

- [15] The SciPy community. gaussian_kde. URL https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.gaussian_kde.html.
- [16] Pedro Domingos and Michael Pazzani. On the optimality of the simple bayesian classifier under zero-one loss. 29:103–130. doi: 10.1023/A:1007413511361.
- [17] Donald F. Specht. Probabilistic neural networks. 3(1):109–118. doi: 10.1016/0893-6080(90)90049-Q.
- [18] Scott S. Sheppard and David C. Jewitt. An abundant population of small irregular satellites around jupiter. *Nature*, 423:261–263, 2003.
- [19] Andrew Y. Ng and Michael I. Jordan. On discriminative vs. generative classifiers: A comparison of logistic regression and naive bayes. In *Advances in Neural Information Processing Systems 14*, pages 841–848. MIT Press, 2001.
- [20] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer. ISBN 978-0-387-31073-2. doi: 10.1007/978-0-387-45528-0. Section 4.3.2, pp. 205–206.
- [21] The scikit-learn developers. Logisticregression, . URL https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [22] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth International Group, Belmont, California, 1984.
- [23] Leo Breiman. Random forests. 45(1):5–32. doi: 10.1023/A:1010933404324.
- [24] The scikit-learn developers. Randomforestclassifier, . URL <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>.
- [25] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. 29(5):1189–1232. doi: 10.1214/aos/1013203451.
- [26] The scikit-learn developers. Histgradientboostingclassifier, . URL <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html>.
- [27] The scikit-learn developers. Ensembles: Gradient boosting, random forests, bagging, voting, stacking, . URL <https://scikit-learn.org/stable/modules/ensemble.html>.
- [28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas K’opf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035. Curran Associates, Inc., 2019.
- [29] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 315–323. PMLR, 2011. URL <https://proceedings.mlr.press/v15/glorot11a.html>.
- [30] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [31] Wang Sang Koon, Martin W Lo, Jerrold E Marsden, and Shane D Ross. Heteroclinic connections between periodic orbits and resonance transitions in celestial mechanics. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 10(2):427–469, 2000.